

# Clean software development

## Day 5

Alessandro Sciarra

Z02 – Software Development Center

10 October 2025

# Topics of the day

- 1 Code review
- 2 Review another project
- 3 Finalise your review
- 4 Sum up reviews

## Our last day together

All good things come to an end. Everything you learnt this week should set up a new begin, though!

# Code review

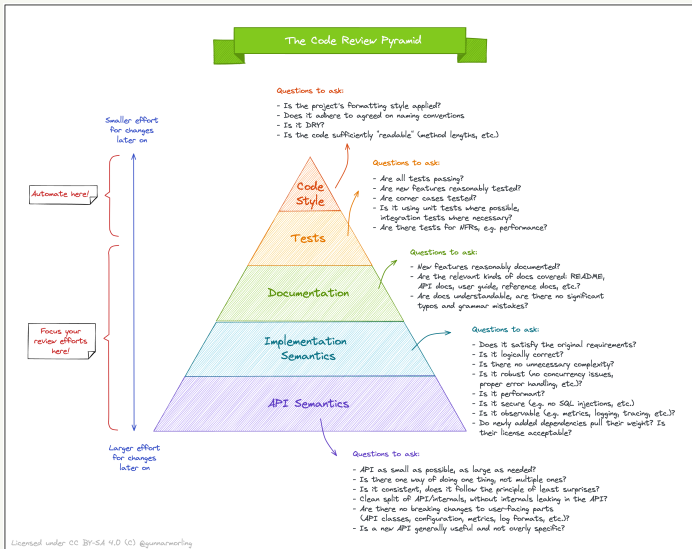
# On a technical level

- 1 Focus on important things, first
- 2 Never forget documentation as part of the review
- 3 If needed, use tools to assist you { Static code analysis, AI, etc. }
  - ↳ your brain is the ultimate tool, though
- 4 Often just reading the code is not enough
  - ↳ isolate minimal working examples (MWE) when needed
  - ↳ consider portability, if appropriate { E.g. run on different OS }
  - ↳ interact with developers in your team
- 5 Keep in mind basically everything we discussed this week 😊

A code review is an investment for your project!



# On a technical level



# On a more abstract level

Some aspects to keep in mind

- 1 Always try to find something positive
- 2 Listen to your code sense (in the right way)
- 3 Do not be biased from who wrote the code
- 4 Try to schedule code reviews as early as possible
- 5 Consider a code review as an opportunity
  - ↳ for both who wrote the code and for the reviewer(s)

We cover for each other

«I expect that, if I have got a team of 15 people and I decide I have to divide that team in half and take 7 people this way and 8 people that way, the 8 people remaining will still be able to operate.»

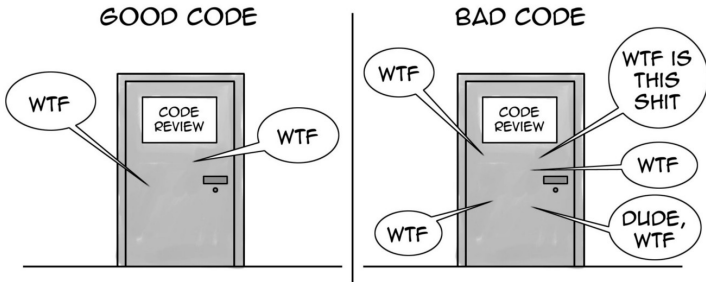
Uncle Bob as CTO



# Do not get emotional...



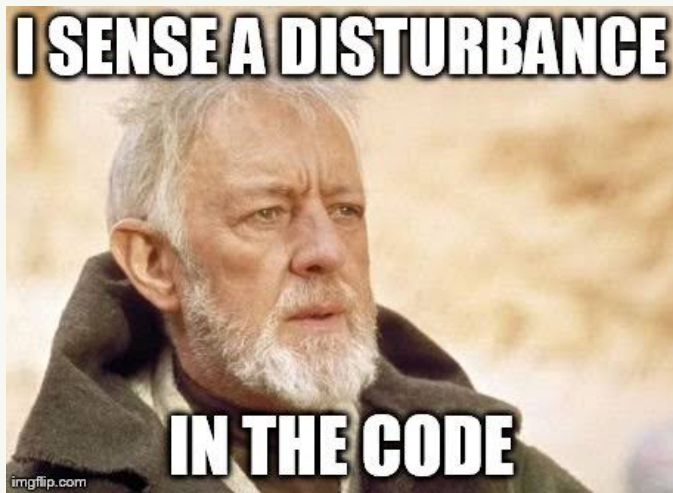
# ...but listen to yourself!



THE ONLY VALID MEASUREMENT OF CODE QUALITY: WTFs/MINUTE



...but listen to yourself!



# Receiving a code review

- The code review evaluates your code, not you
- It is meant to make the code **even** better
- You and reviewer are part of the same team
- Remember that code review is a dialogue
  - ↳ It is an opportunity to learn for everybody

 How to write reviews your team looks forward to?

A nice write-up worth reading with excellent guidance for both authors and reviewers.



# A last comment for development in academia

**If you never did a code review in your group**

Do not feel bad. That's not surprising (for me). Consider initiating this habit, if reasonably possible.



# A last comment for development in academia

## If you never did a code review in your group

Do not feel bad. That's not surprising (for me). Consider initiating this habit, if reasonably possible.

## If you use code reviews in your group

That's great. Be greedy to be a reviewer, even if the developer who wrote the code is much more senior than you. A code review should not become a pedagogical lecture, but asking (an appropriate amount of) clarifications questions makes it an excellent opportunity to grow.



# A last comment for development in academia

## If you never did a code review in your group

Do not feel bad. That's not surprising (for me). Consider initiating this habit, if reasonably possible.

## If you use code reviews in your group

That's great. Be greedy to be a reviewer, even if the developer who wrote the code is much more senior than you. A code review should not become a pedagogical lecture, but asking (an appropriate amount of) clarifications questions makes it an excellent opportunity to grow.

## I code alone, what's my take on this?

You'll need to be stricter with yourself. Never miss an opportunity to implement improvements.

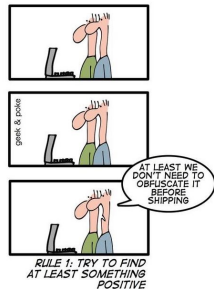


Review another project

# Review another project

- 1 Take the project of another group { from yesterday }
  - ↳ you will review it as a group
  - ↳ you have ~60 minutes time
- 2 Have a rough look to the whole code { Alone }
  - ↳ note down your comments and questions
- 3 Do the review together as a group
  - ↳ a simple Markdown file is enough
  - ↳ stick to the principles we discussed today
  - ↳ classify your comments as major or minor

## HOW TO MAKE A GOOD CODE REVIEW



Break

**Finalise your review**

# Finalise your review

- 4 Finalise the review to present it to the whole class in ~10 minutes
- Start from the positive aspects
  - Did you learn/discover anything during the review?
  - How difficult was to get started reading the code overall?
  - Try to relate your comments with the principles we learnt



Sum up reviews

# Sum up reviews

It is an opportunity to reflect on what we learnt over the week

- 1 Each group presents its review in 10 minutes
- 2 After the presentation we have few minutes for discussions
  - ↳ Focus on clean code principles



Lunch break

# Clean software development

## Feedback and closing

Alessandro Sciarra

David A. Clarke

**Thank you!**

# Wrapping up (part of) the advice

- 1 Clean software development requires discipline
- 2 Never be lazy! { Either do or leave trace (e.g. an issue) }
- 3 Every day you have the possibility to become a little better
- 4 Tests (and documentation) are part of the code
- 5 Almost all principle belong to all languages
- 6 Your **code sense** will develop and guide you
- 7 Changing mind is positive and part of the game



# Wrapping up (part of) the advice

- 1 Clean software development requires discipline
- 2 Never be lazy! { Either do or leave trace (e.g. an issue) }
- 3 Every day you have the possibility to become a little better
- 4 Tests (and documentation) are part of the code
- 5 Almost all principle belong to all languages
- 6 Your **code sense** will develop and guide you
- 7 Changing mind is positive and part of the game

## Moral of the story

A young violinist got lost on his way to a performance. He stopped an old man on the corner and asked him how to get to Carnegie Hall. The old man looked at the violinist and the violin tucked under his arm and said: «Practice, son. Practice!»

R. C. Martin



# A first feedback: How do you feel?



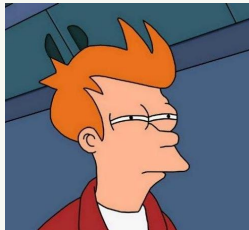
# A first feedback: How do you feel?



# A first feedback: How do you feel?



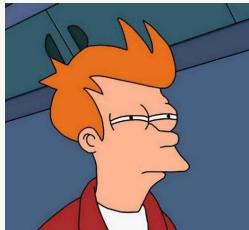
# A first feedback: How do you feel?



# A first feedback: How do you feel?



# A first feedback: How do you feel?



# A first feedback: How do you feel?



# A short Git exercise

- 1 Open your terminal in the common repository
- 2 Create a file with a **unique** filename in the  Feedback folder  
↳ E.g. `touch feedback_$(date +%s.%3N)`
- 3 Write a feedback sentence into such a file
- 4 Anonymously commit and push

**GIT\_AUTHOR\_NAME** is the human-readable name in the “author” field.

**GIT\_AUTHOR\_EMAIL** is the email for the “author” field.

**GIT\_AUTHOR\_DATE** is the timestamp used for the “author” field.

**GIT\_COMMITTER\_NAME** sets the human name for the “committer” field.

**GIT\_COMMITTER\_EMAIL** is the email address for the “committer” field.

**GIT\_COMMITTER\_DATE** is used for the timestamp in the “committer” field.



# Open final discussion

- Do you have any further question?
- Would you like to share any thought?
- Is there anything you disliked and would change?



# And now?



«And suddenly you know: It's time to start something new and trust the magic of beginnings.»

**Meister Eckhart**